Computers & Industrial Engineering 222 (2026) 112340

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

Integrating human knowledge through action masking in reinforcement learning for operations research

Mirko Stappert [a], Bernhard Lutz [b,*], Niklas Goby [a], Dirk Neumann [a]

[a] University of Freiburg, Rempartstr. 16, 79098 Freiburg, Germany
[b] University of Vienna, Nordbergstr. 1, 1090 Vienna, Austria

ARTICLE INFO

Dataset link: https://doi.org/10.6084/m9.figshare.33098051

Keywords:
Reinforcement learning
Human domain knowledge
Action masking
Scheduling
Load management
Inventory management

ABSTRACT

Reinforcement learning (RL) provides a powerful method to address problems in operations research. However, its real-world application is often hindered by limited acceptance and adoption among practitioners. One important reason is that learned policies may appear opaque or misaligned with existing domain knowledge. In this study, we analyze the benefits and caveats of including human knowledge via action masking. While action masking has so far been used to exclude invalid actions, its ability to integrate human expertise remains underexplored. Human expertise is often expressed in the form of heuristics that suggest reasonable, near-optimal actions in specific situations. Incorporating such heuristics into the RL policy may improve the practical usability of RL systems by aligning them more closely with human expectations. Yet, enforcing heuristic actions too strictly may limit exploration and prevent the policy from learning optimal actions, potentially resulting in lower overall performance. We analyze the effects of action masking based on three problems with different characteristics, namely, paint shop scheduling, peak load management, and inventory management. Our findings demonstrate that incorporating human knowledge through action masking can achieve substantial improvements over policies trained without action masking. In addition, we find that action masking is crucial for learning effective policies in constrained action spaces, where certain actions can only be performed a limited number of times. Finally, we highlight the potential for suboptimal outcomes when action masks are overly restrictive.

1. Introduction

Reinforcement Learning (RL), a branch of machine learning, has been successfully applied to solve complex problems such as the games of Chess and Go by self-play only (Silver et al., 2018, 2017). The resulting performance even surpasses the skills of human experts and traditional AI engines. In RL, an agent interacts with an environment to maximize the expected reward (Sutton & Barto, 2018). The environment presents the current state to the agent, who then decides upon the action to perform next. After receiving an action, the environment transforms to the next state and emits a numerical reward to the agent. These achievements naturally raise the question of how RL can be employed in operations research (OR).

Over the past few years, several studies developed RL solution methods for OR problems. The considered problem areas include production planning and control (Esteso et al., 2023; Panzer & Bender, 2022; Zhang & Zhu, 2024), supply chain management and inventory management (Rolf et al., 2023), machine scheduling (Chen et al., 2024; Hou et al., 2025; Kayhan & Yildiz, 2023), machine maintenance planning (Goby et al., 2023; Ogunfowora & Najjaran, 2023), and quality control (Paraschos & Koulouriotis, 2024), among many others. A major advantage of RL over traditional solution methods (e.g., hand-crafted rules, heuristics, mathematical solvers) is that a trained RL policy presents a fast and adaptive solution method. Businesses often encounter similar decision problems where the parameters follow known distributions like package delivery problems in the same district (e.g., Bengio et al., 2021; Schiewe & Stinzendörfer, 2024), or sequencing problems with similar demand plans (e.g., Brammer et al., 2022a, 2022b). A RL policy can be trained in a simulated environment and subsequently transfer knowledge from prior learning experiences to unseen problem instances (Taylor & Stone, 2009). By contrast, the aforementioned traditional solution methods are designed to solve each problem instance individually without relying on prior experience. In addition, RL can be trained to account for the uncertainty in real-world operations regarding delivery times or customer demand. The paradigm

* Corresponding author.
E-mail addresses: stappertmirko@gmail.com (M. Stappert), bernhard.lutz@univie.ac.at (B. Lutz), niklas.goby@googlemail.com (N. Goby), dirk.neumann@is.uni-freiburg.de (D. Neumann).

https://doi.org/10.1016/j.cie.2026.112340
Received 8 July 2025; Received in revised form 27 July 2026; Accepted 23 August 2026
Available online 2 September 2026
0360-8352/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

of RL hence aligns particularly well with the goals of Industry 4.0, where adaptive and fast decision-making under uncertainty are crucial (Choi et al., 2022; Weckenborg et al., 2024).

A major challenge of implementing RL in real-world operations is the task of converting the problem presumably formulated as a mixed-integer linear programming (MILP) problem to a Markov decision process, including states, actions, and reward function, that is suitable for RL policy learning (Sutton & Barto, 2018). Here, businesses need to ensure that (i) the policy generates valid solutions, and that (ii) a (near-)optimal policy of the MDP also generates (near-)optimal solutions of the actual MILP problem. A common approach to guiding the RL policy towards generating only valid solutions is given by returning the same state and penalizing invalid actions with large negative rewards (e.g., Brammer et al., 2022b; Valet et al., 2022). While this method generally achieves the desired outcome, the policy will still perform invalid actions during the training process, thereby increasing the time required for learning effective policies. An alternative is given by action masking (e.g., Huang & Ontañón, 2022; Luo et al., 2022; Wang et al., 2024), a technique in RL that constrains the action space in specific states.

In addition to these technical challenges, the implementation of RL, like other AI-based solution methods, often fails in real-world operations due to limited acceptance and adoption by human decision-makers (Dietvorst et al., 2015; Lehmann et al., 2022). Although problems from OR are often considered as repetitive and low-stakes (Notz & Pibernik, 2024), practitioners still seek to retain control over decisions and the ability to make adjustments, particularly when models produce unsatisfactory outcomes (Dietvorst et al., 2018). Therefore, a promising approach to increase acceptance and practical usability is to allow managers to modify RL policies by incorporating human domain knowledge. Human knowledge can prescribe reasonable heuristic actions in certain states, or even provably optimal actions. Ensuring that a policy reflects such domain expertise can make model behavior more aligned with established decision processes, thereby facilitating its integration into practice and increasing the likelihood that decision-makers rely on its recommendations.

In this study, we analyze the inclusion of human domain knowledge via action masking. While traditionally used to prevent invalid actions, its potential for enforcing heuristic-suggested or provably optimal actions remains unexplored. Although a RL policy could, in theory, learn the actions suggested by the action mask implicitly after sufficient training time, there is no such guarantee in DRL due to the "deadly triad" of function approximation, bootstrapping, and off-policy learning (Sutton & Barto, 2018; Van Hasselt et al., 2018). Unlike tabular-based Q-learning, where convergence to a global maximum is theoretically guaranteed (Sutton & Barto, 2018), DRL approximates the values of states using a neural network, introducing function approximation errors. In fact, an update of the network's weights influences several states instead of only a single state–action entry of the Q-value table. In addition, policy updates depend on previous estimates (bootstrapping) and are often applied to past experience (off-policy learning), thereby possibly amplifying approximation errors. Including action masks in the training process could hence lead to superior policies. At the same time, action masks might also restrict the flexibility of the policy, preventing it from reaching particular optima.

We examine the benefits and caveats of action masking based on three OR problems with different characteristics (paint shop scheduling, peak load management, inventory management). In particular, we outline the effects of action masking when incorporating different types of human knowledge regarding heuristic and provably optimal actions. Our evaluation yields several important insights. First, we find that action masking can lead to significant increases in the performance of a trained RL policy. Enforcing heuristic actions in a paint shop scheduling problem significantly decreases color changes compared to unmasked RL. Second, we observe that, for problems with constrained action spaces, applying action masking can even be necessary to learn effective policies. The considered peak load management only allows a limited number of turn-off operations. If the policy is not guided using human domain knowledge, it never learned to effectively manage the load. Third, we find that action masking should also be applied with caution as the enforcement of non-optimal heuristics may harm the solution quality.

Our study contributes to the literature by analyzing the effects of incorporating human domain knowledge into the training and application of RL policies via action masking. Alternatives to action masking include imitation learning (Hussein et al., 2017) and inverse RL (Arora & Doshi, 2021), allowing the policy to learn from observing human actions. However, it may not often be possible to learn from human actions due to labor shortage or the problem is simply too complex for a human to solve optimally. Another related research stream is RL from human feedback (RLHF, Griffith et al., 2013; Kaufmann et al., 2023), where the RL agent learns from human feedback instead of a reward function. Unlike RLHF, which relies on indirect and possibly noisy preference signals to perform policy updates (Casper et al., 2023), action masking directly integrates human domain knowledge into the policy, resulting in faster learning and greater safety in constrained environments. Other approaches employ reward shaping (Cheng et al., 2021) or transfer learning (Bianchi et al., 2015). Yet, these methods still do not allow human decision-makers to enforce or disallow actions in certain situations. While our study is limited to offline learning, where a policy is pretrained in a simulated environment, action masking may also help to counter the cold start problem that occurs when RL is trained directly in real-world applications. Here, action masking can be used to prevent the execution of clearly unreasonable actions, as, e.g., reordering stock when the current inventory is full, which would result in substantial losses. We thus hope to pave the way for future studies that develop RL-based solution methods for OR problems.

The remainder of this paper is structured as follows. Section 2 describes the methodological framework by introducing definitions of Markov decision processes and detailing how human domain knowledge can be included via action masking. Section 3 presents the paint shop scheduling problem, utilizing action masking to implement multiple heuristics. Section 4 explores the peak load management problem with constrained actions. Section 5 examines an inventory management problem with stochastic demands and delivery times. Section 6 discusses our findings and provides an outlook on future research.

2. Methodological framework

2.1. Reinforcement learning

Reinforcement learning is a type of machine learning where an agent interacts with an environment to learn a policy, which specifies the optimal action(s) in a given state. Most real-world decision problems with known parameters and a finite number of actions can be transferred to a problem from RL. RL problems are formally modeled as a Markov decision process (Sutton & Barto, 2018).

Definition 1 (*Markov Decision Process*). A Markov decision process (MDP) is a tuple $(S, A, T, R, s_0, \gamma)$ with state space S, action space A, transition function $T(s_{t+1} \mid a, s_t) \in [0, 1]$, reward function $R(r_t \mid a, s_t) \in [0, 1]$, initial state s_0 and discount factor $\gamma \in [0, 1]$.

The state $s_t \in S$ reflects the current situation of the environment. The action space A denotes all possible decision options (e.g., ordering supplies, producing a certain item). The transition function $T(s_{t+1} \mid a, s_t)$ specifies how the environment evolves from one state to another if a particular action is performed. The transition function can be deterministic (e.g., in production scheduling problems with fixed processing times) or stochastic (e.g., in inventory management problems with demand uncertainty). In the general stochastic case, $T(s_{t+1} \mid a, s_t)$ denotes the probability that s_{t+1} follows s_t given that action a is performed. The

reward function can also be stochastic, so that $R(r_t \mid a, s_t)$ specifies the probability of receiving reward r_t when action a is performed in state s_t. The reward function should be designed in a way that assigns positive rewards to actions that are effective in achieving the overarching goal (e.g., fulfilling demand while minimizing storage costs) and negative rewards to ineffective and sub-optimal actions.

The goal of RL is to find a policy $\pi_\theta(a|s_t)$, which maximizes the expected total sum of discounted rewards $J = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{H} \gamma^t r_t\right]$. Specifically, $\pi_\theta(a|s_t)$ denotes the probability of performing action a in state s_t. In deep reinforcement learning, a neural network receives the state as input and outputs a probability distribution over the action space. The policy parameters θ denote the weights of the neural network, which must be learned during the training process. The training process alternates between generating experience in the form of state–action-reward tuples $s_0, a_0, r_0, \ldots, s_T, a_T, r_T$ by sampling from the policy and using this experience to update the policy parameters θ.

So far, it is assumed that the action space is constant in every state, i.e., each action can always be performed. However, in real-world applications, only a subset of all actions are admissible in each specific state. Invalid actions are often excluded by introducing $A(s_t)$ as the set of admissible actions in s_t. While introducing $A(s_t)$ is theoretically feasible, there are practical limitations. In fact, the policy networks employed in deep RL have a fixed number of output neurons, which cannot be adjusted dynamically during the training process. Consequently, invalid actions must be handled manually during policy training and subsequent real-world application. A common approach is to mask invalid actions by setting their probability to zero and sampling from the remaining actions.

While action masking is an obvious step in the application of a trained policy, it also affects the training process in general. First, action masking prevents the exploration of invalid actions, which increases the efficiency of the training process. Second, action masking influences learning by updating the policy parameters according to an adjusted policy gradient (Huang & Ontañón, 2022). Third, action masking provides an early guideline for an untrained RL policy so that it suffers less from the cold start problem, where obviously non-optimal actions are explored in the early learning episodes. From an implementation view, action masking has recently been implemented as part of the popular RL framework "Stable Baselines3" (Raffin et al., 2021) based on the implementation of Huang and Ontañón (2022).

To the best of our knowledge, action masking has so far only been used to exclude invalid actions (e.g., Berner et al., 2019; Huang & Ontañón, 2022; Vinyals et al., 2017). In this study, we aim to analyze the benefits and risks of action masking to include human domain knowledge, namely, (i) prescribe heuristic rules and, more strictly, (ii) enforce provably optimal actions.

2.2. Integrating human domain knowledge via action masking

An action mask m is a function $m\colon S \times A \to \{0, 1\}$ that reduces the set of actions in a given state (Huang & Ontañón, 2022). By evaluating $m(a, s_t)$ for all actions, we obtain

$$A(s_t) = \{a \in A \mid m(a, s_t) = 1\}. \tag{1}$$

Action masking can hence be used to reduce the set of admissible actions in reinforcement learning.

To exclude invalid actions, we simply set $m(a, s_t) = 0$ if a is invalid in s_t (Berner et al., 2019; Huang & Ontañón, 2022; Vinyals et al., 2017)

$$m(a, s_t) = \begin{cases} 1 & \text{if } a \text{ is valid in } s_t \\ 0 & \text{else.} \end{cases} \tag{2}$$

Prescribing domain heuristics

For many problems from operations research, there are certain heuristics that can be used to restrict the action space. For instance, in an inventory management problem, there might be knowledge that we should produce at most twice the demand of the previous period. While producing more is still possible, we know that many of the products will not be sold in the next period. Therefore, the heuristic suggests that the action space is restricted to actions that order less than twice the demand of the previous period.

However, employing action masking to incorporate human domain knowledge can lead to a bias–variance trade-off. By reducing the action space, masking acts as a form of regularization, which typically lowers variance by stabilizing learning. At the same time, it may introduce approximation bias if optimal actions are excluded. In an idealized setting, where the mask only removes suboptimal actions, both bias and variance decrease. Conversely, if masking strategies rely on heuristics and domain knowledge, they are unlikely to stipulate an optimal policy. In such cases, masking reduces variance at the cost of increased bias. Designing masking strategies therefore requires balancing domain knowledge with sufficient learning flexibility, as detailed in the following.

Let $h(s_t)$ denote the action suggested by the given heuristic in state s_t. We do not strictly enforce a heuristic, as this would make the policy equal to the heuristic. Instead, we aim to approximately prescribe the heuristic, thereby imparting knowledge to the RL agent but also giving it the flexibility to deviate from the action suggested by the heuristic.

Prescribing a heuristic while providing the RL policy with a limited amount of flexibility is, in particular, possible if the action space has a certain structure that allows us to rank actions based on a reasonable order. For instance, in inventory management problems, this corresponds to the quantity of items to be produced in the next period. Given such an order of actions, we can restrict the action space to the action suggested by the heuristic and the actions close to $h(s_t)$ according to a given threshold M

$$m(a, s_t) = \begin{cases} 1 & \text{if } \mid h(s_t) - a \mid \leq M \\ 0 & \text{else.} \end{cases} \tag{3}$$

Similarly, the action space can be restricted to actions greater or smaller than the action suggested by the heuristic

$$m(a, s_t) = \begin{cases} 1 & \text{if } a \geq h(s_t) \text{ (conversely } a \leq h(s_t)) \\ 0 & \text{else.} \end{cases} \tag{4}$$

Enforcing optimal actions

An action a^* is optimal in s_t if there is an action sequence that maximizes the cumulative reward in s_t and starts with a^*. Optimal actions should thus be allowed while excluding all non-optimal actions. In general, we might know the optimal actions for a small subset of states $S' \subset S$. We thus set $m(a^*, s_t) = 1$ for all optimal actions a^* and $m(a, s_t) = 0$ for all non-optimal actions in all $s_t \in S'$. For all $s_t \in S \setminus S'$, where optimal actions are not known, we allow all actions. Together, we get

$$m(a, s_t) = \begin{cases} 1 & \text{if } s_t \in S' \text{ and } a \text{ is optimal in } s_t \\ 0 & \text{if } s_t \in S' \text{ and } a \text{ is not optimal in } s_t \\ 1 & \text{if } s_t \in S \setminus S'. \end{cases} \tag{5}$$

Note that optimality of the original MDP is preserved as long as the employed masks do not exclude optimal actions.

Combining action masks

So far, we only discussed individual action masks. However, there might be situations where several reasonable action masks should be combined. Therefore, we outline two approaches to combine action masks. First, we define the conjunction operation $m_1 \oplus m_2$, which only allows action a to be performed in state s_t, if both masks allow a

$$(m_1 \oplus m_2)(a, s_t) = \begin{cases} 1 & \text{if } m_1(a, s_t) = 1 \text{ and } m_2(a, s_t) = 1 \\ 0 & \text{else.} \end{cases} \tag{6}$$

The second operation $m_1 \circledcirc m_2$ applies two action masks in a sequential way, while assigning higher priority to m_1. This approach is particularly relevant if we want to prioritize one mask over another. $m_1 \circledcirc m_2$ thus applies m_1 if m_1 is *active* in s_t, i.e., m_1 forbids at least one action, and m_2 otherwise

$$(m_1 \circledcirc m_2)(a, s_t) = \begin{cases} m_1(a, s_t) & \text{if } \exists a' \in A\colon\ m_1(a', s_t) = 0 \\ m_2(a, s_t) & \text{else.} \end{cases} \tag{7}$$

Policy gradient calculations with action masking

The implementation of action masking by Huang and Ontañón (2022) builds upon the policy gradient method. The objective of maximizing the expected sum of discounted rewards $J = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{H} \gamma^t r_t\right]$ is achieved by applying gradient ascent optimization to the function J with respect to the policy parameters θ. The calculation of the policy gradient is enabled by the policy gradient theorem (Sutton & Barto, 2018)

$$\nabla_\theta J = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{H} \nabla_\theta \log \pi_\theta(a|s_t) G_t\right], \tag{8}$$

where $G_t = \sum_{k=0}^{H} \gamma^k r_{t+k}$ is the sum of discounted rewards starting at time step t. The gradient $\nabla_\theta J$ can be estimated by sampling trajectories from the policy π_θ and averaging the resulting values inside the expectation operator. The action probabilities are calculated via the softmax function from the logits $l_\theta(a|s_t)$ as

$$\pi_\theta(a|s_t) = [\text{softmax}(l_\theta(\cdot|s_t))]_a = \frac{\exp(l_\theta(a|s_t))}{\sum_{a' \in A} \exp(l_\theta(a'|s_t))} \tag{9}$$

Given an action mask m, one proceeds as follows. First, the logits are updated by setting actions with $m(a, s_t) = 0$ to negative infinity

$$l_\theta^m(a|s_t) = \begin{cases} l_\theta(a|s_t) & \text{if } m(a, s_t) = 1 \\ -\infty & \text{if } m(a, s_t) = 0. \end{cases} \tag{10}$$

Second, the action probabilities are recalculated based on the adjusted logits

$$\pi_\theta^m(a|s_t) = [\text{softmax}(l_\theta^m(\cdot|s_t))]_a. \tag{11}$$

The effect of action masking can easily be shown on a small example. Let the logits of state s_t be given as $[l_\theta(a_0|s_t), l_\theta(a_1|s_t), l_\theta(a_2|s_t)] = [1, 1, 1]$. The three actions a_0, a_1, a_2 are thus performed with equal probability of $\frac{1}{3}$. Given an action mask m that disallows action a_2 in s_t, the resulting probabilities based on the adjusted logits are $\pi_\theta^m(a_0|s_t) = \pi_\theta^m(a_1|s_t) = \frac{1}{2}$ and $\pi_\theta^m(a_2|s_t) = 0$.

Finally, the modified policy gradient $\nabla_\theta^m J$ is calculated using the updated action probabilities of $\pi_\theta^m(a|s_t)$

$$\nabla_\theta^m J = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{H} \nabla_\theta \log \pi_\theta^m(a|s_t) G_t\right]. \tag{12}$$

An action mask is thus not only enforced by hand but also incorporated into the learning process itself by performing gradient ascent steps with an adjusted policy gradient(see Fig. 1).

3. Problem 1: Paint shop scheduling

The paint shop scheduling problem deals with minimizing the number of color changes in the painting procedure of the automotive manufacturing process.[1] Therefore, an incoming sequence of unordered cars with assigned colors has to be reshuffled using a multi-lane buffer system that allows storage and retrieval operations. An exemplary buffer system including the notation is shown in Fig. 1.

[1] The results presented in this section are subset of a larger study called "Solving the Paint Shop Problem with Flexible Management of Multi-Lane Buffers Using Reinforcement Learning and Action Masking" (Stappert et al., 2026). We believe that including a short excerpt of the results assists in demonstrating the potential benefits of action masking.

3.1. Environment

At each time step, the next car of the incoming sequence can be stored in the rightmost free position of a buffer lane or the rightmost car of a buffer lane can be retrieved and added to the last position of the outgoing sequence. The buffer consists of L lanes, each of width W. The action space hence consists of $2L$ actions

$$A^{\text{PS}} = \{\underbrace{1, \dots, L}_{\text{Retrieve actions}}, \underbrace{L+1, \dots, 2L}_{\text{Store actions}}\}. \tag{13}$$

We define the state to contain the entire buffer content, the next $K = 5$ colors of the incoming sequence, and the current color of the outgoing sequence

$$s_t^{\text{PS}} = (\underbrace{B_{t,1,1}, \dots, B_{t,L,W}}_{\text{Buffer content}}, \underbrace{e_{t,1}, \dots, e_{t,K}}_{\text{K next incoming cars of input sequence}}, \underbrace{p_t}_{\text{Current painting color}}). \tag{14}$$

Each value $(0, 1, \dots, C)$ is one-hot encoded and either denotes an empty position or a color $1, \dots, C$.

The reward function penalizes retrieve actions causing a color change. Invalid actions get a reward of -10. Valid store actions and retrieve actions that cause a color change receive a zero reward. However, valid retrieve actions not causing a color change are assigned a reward of 1.

$$R^{\text{PS}}(a, s_t) = \begin{cases} 0 & a \le L,\ B_{t,a,W} \neq 0,\ B_{t,a,W} \neq p_t \\ & \quad \text{(valid retrieval with color change)} \\ 1 & a \le L,\ B_{t,a,W} \neq 0,\ B_{t,a,W} = p_t \\ & \quad \text{(valid retrieval without color change)} \\ -10 & a \le L,\ B_{t,a,W} = 0 \quad \text{(invalid retrieval)} \\ 0 & a > L,\ B_{t,a-L,1} = 0,\ e_{t,1} \neq 0 \quad \text{(valid store)} \\ -10 & a > L,\ B_{t,a-L,1} \neq 0 \text{ or } e_{t,1} = 0 \quad \text{(invalid store)} \end{cases} \tag{15}$$

3.2. Action masking

We consider four action masks as illustrated in Fig. 2. The first action mask m^{INV} excludes invalid actions, i.e., storing in a full lane or retrieving from an empty lane

$$m^{\text{INV}}(a, s_t) = \begin{cases} 1 & \text{if } a \in \{1, \dots, L\},\ B_{t,a,W} \neq 0 \\ 1 & \text{if } a \in \{L+1, \dots, 2L\},\ B_{t,a-L,1} = 0, \\ & \quad e_{t,1} \neq 0 \\ 0 & \text{else.} \end{cases} \tag{16}$$

Adding this action mask hence waives the need to penalize invalid actions with negative rewards, which should lead to a more efficient learning process.

The next two action masks m^{GR} and m^{FT} enforce so called *greedy retrieval* and *fast-track*. These provably optimal actions are not possible in every state. We thus define two action masks that enforce these actions whenever they are possible, following the general procedure of enforcing optimal actions (see). An action is called *greedy retrieval* if it retrieves a car from the rightmost position of a buffer lane that has the same color as the current color of the outgoing sequence. The greedy retrieval mask $m^{\text{GR}}(a, s_t)$ is given as

$$m^{\text{GR}}(a, s_t) = \begin{cases} 1 & \text{if } a \in \{1, \dots, L\}, p_t = B_{t,a,W} \\ 1 & \text{if } p_t \neq B_{t,i,W}\ \forall i = 1, \dots, L \\ 0 & \text{else.} \end{cases} \tag{17}$$

We refer to *fast-track* as storing a car in an empty buffer lane in order to directly retrieve in the next step without causing a color change. The fast-track mask m^{FT} enforces store actions to empty lanes if a

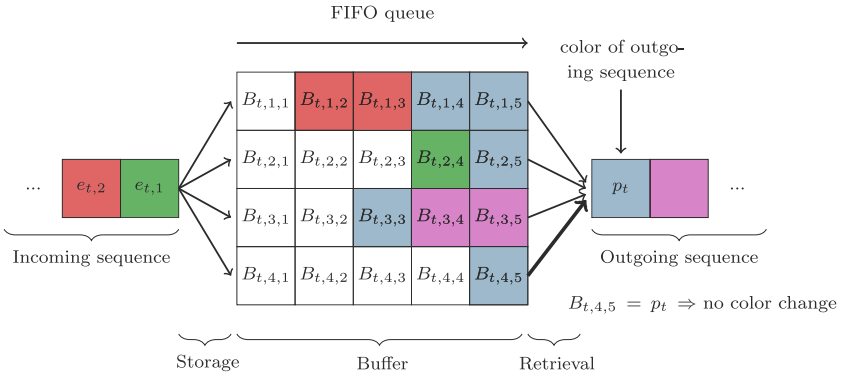
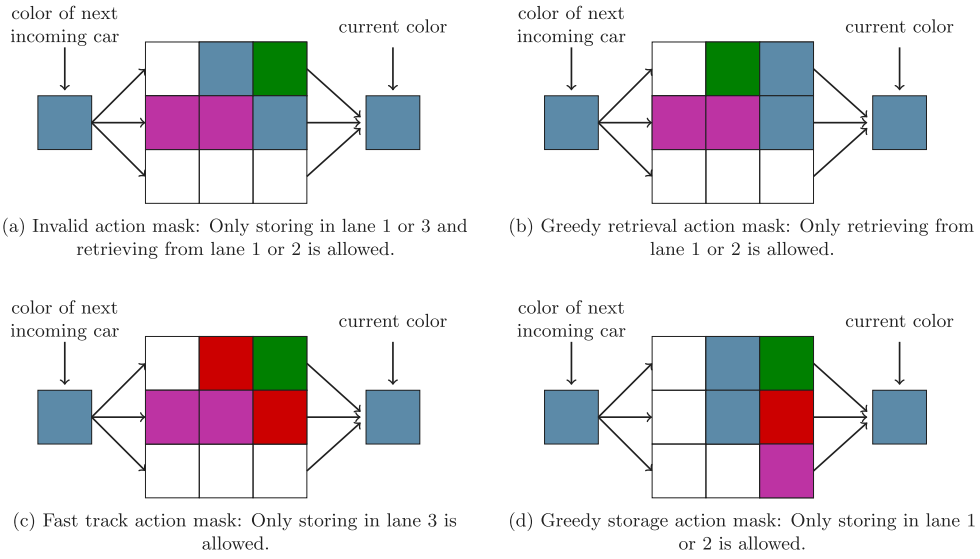


Fig. 1. Paint shop problem with a 4 × 5 buffer (four lanes of width five). The system retrieves from lane 4 without causing a color change.



(a) Invalid action mask: Only storing in lane 1 or 3 and retrieving from lane 1 or 2 is allowed.

(b) Greedy retrieval action mask: Only retrieving from lane 1 or 2 is allowed.

(c) Fast track action mask: Only storing in lane 3 is allowed.

(d) Greedy storage action mask: Only storing in lane 1 or 2 is allowed.

Fig. 2. Illustration of the four considered action masks.

subsequent greedy retrieval from this lane is possible. Otherwise, the mask allows all actions if fast-track is not possible

$$m^{\mathrm{FT}}(a, s_t) = \begin{cases} 1 & \text{if } a \in \{L+1, \dots, 2L\},\ B_{t,a-L,W} = 0,\ p_t = e_{t,1} \\ 1 & \text{if } p_t \neq e_{t,1} \text{ or } B_{t,i,W} \neq 0,\ \forall i = 1, \dots, L \\ 0 & \text{else.} \end{cases} \tag{18}$$

Finally, we consider *greedy storage*, i.e., storing the current car from the incoming sequence in a buffer lane, where the leftmost car is of the same color. While we cannot prove that the heuristic is indeed optimal, it still presents a reasonable heuristic. However, our action space does not possess any structure that follows a reasonable order as colors are uniquely different. We therefore cannot follow Eq. (3) or Eq. (4) to approximately prescribe the heuristic. Nevertheless, we still evaluate a fourth action mask $m^{\mathrm{GS}}(a, s_t)$ that enforces greedy storage

$$m^{\mathrm{GS}}(a, s_t) = \begin{cases} 1 & \text{if } a \in \{L+1, \dots, 2L\},\ \exists j \in \{2, \dots, W\}: \\ & \quad e_{t,1} = B_{t,a-L,j},\ B_{t,a-L,j-1} = 0 \\ 1 & \text{if } \nexists i \in \{1, \dots, L\},\ j \in \{2, \dots, W\}: \\ & \quad e_{t,1} = B_{t,i,j},\ B_{t,i,j-1} = 0 \\ 0 & \text{else.} \end{cases} \tag{19}$$

Note that employing the aforementioned action masks does not solve the paint shop problem. In fact, greedy retrieval, greedy storage, and fast-track actions are often not possible. Therefore, an effective solution policy still needs to perform several optimal storage and retrieval actions.

So far, we considered several action masks independently. However, we can combine multiple action masks and perform our evaluations using the procedure for combining action masks defined in Eqs. (6) and (7). We give the least priority to m^{GS}, as we do not know whether greedy storage is indeed optimal

- $m^{\mathrm{INV}} \oplus \left(\left(m^{\mathrm{GR}} \otimes m^{\mathrm{FT}} \right) \otimes m^{\mathrm{GS}} \right)$ (all masks)
- $m^{\mathrm{INV}} \oplus \left(m^{\mathrm{GR}} \otimes m^{\mathrm{FT}} \right)$ (invalid + greedy retrieval + fast-track)
- $m^{\mathrm{INV}} \oplus m^{\mathrm{GR}}$ (invalid + greedy retrieval)
- m^{INV} (invalid)

3.3. Results

We train only a single RL policy for each configuration as the state transitions are fully deterministic. Each RL policy is provided 5 min to generate solutions via policy sampling. We then compare the number of color changes of RL with different action masks against several baselines. First, we include a greedy heuristic that performs greedy retrieval, greedy storage and fast-track actions whenever possible. If

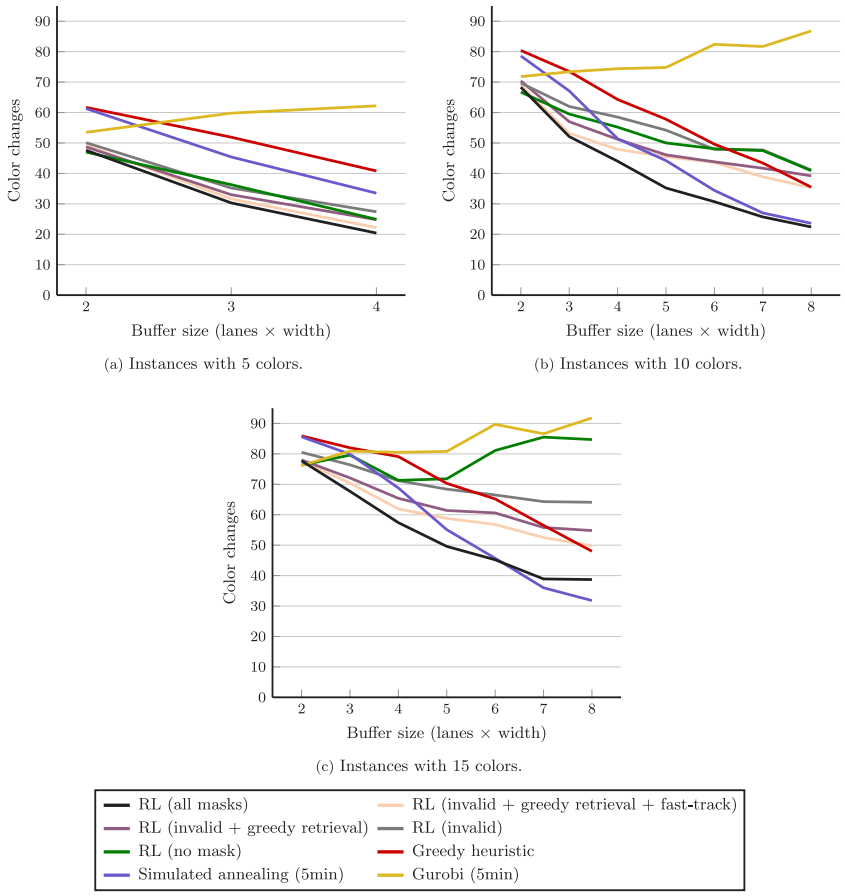


Fig. 3. Evaluation results (color changes) for all RL approaches and Greedy heuristic.

none of these are possible, it performs a deterministic fallback action based on a fixed tie-breaking rule (i.e., store in the lane with the fewest stored cars, or retrieve from the lane with the highest-index color). Second, we include simulated annealing with geometric cooling based on the greedy solution. The neighborhood is given by all permutations within sequences of exclusive storage and retrieval operations. Third, we implement Gurobi using "gurobipy" version 10.0.2 based on the integer linear program formulation of the problem. Both simulated annealing and Gurobi are also provided five minutes per instance to search for a solution.

The problem instances have $C = 5, 10, 15$ colors and buffer sizes from 2×2 to 8×8. For each buffer size, we generate ten random incoming sequences of length 100. Each sequence is randomly generated by sampling the color at each sequence position independently and with equal probability. Note that we only consider problems with fewer buffer lanes than colors.

The results presenting the number of color changes are shown in Fig. 3. We find that the benefits of action masking in reducing color changes depend on the buffer size. For 2×2 buffers, the reduction in color changes is negligible. However, the difference in color changes increases for larger buffer sizes, especially in the problem instances with 10 and 15 colors. In particular for problems with 8×8 buffers, RL with all action masks causes approximately half the number of color changes compared to RL without any action masking (22.4 vs 40.9 in 10-color problems and 38.7 vs 84.7 in 15-color problems). The best baseline approach is given by simulated annealing, which even outperforms RL with all masks on the large problem instances with 15 colors and 7×7, 8×8 buffers. Greedy and Gurobi do not show competitive results.

We perform Friedman tests (Friedman, 1937) followed by pairwise Nemenyi post-hoc tests (Nemenyi, 1963) for each configuration (number of colors, buffer size) to assess whether the differences in color changes are statistically significant. For each configuration, the Friedman test is significant with $p < 0.001$. We briefly summarize the pairwise results of the Nemenyi tests, focusing on RL (all masks) as the reference. The complete test results with $p < 0.05$ are provided in Appendix B of the supplementary material.

On the 5-color problems, RL (all masks) outperforms vs Greedy with $p < 0.05$ on all buffer sizes. RL (all masks) vs RL (no mask) is significant with $p < 0.05$ on 3×3 buffers. RL (all masks) vs RL (invalid mask) is significant with $p < 0.05$ on 4×4 buffers. The performance differences between RL (all masks) and RL (invalid + greedy retrieval), RL (invalid + greedy retrieval + fast-track) are not significant.

On the 10-color and 15-color problems, RL (all masks) outperforms Greedy with $p < 0.05$ on all buffer sizes except 8×8. Gurobi is outperformed with $p < 0.001$ on all problems with buffers greater or equal to 3×3. Simulated annealing is only outperformed with $p < 0.05$ on problems with 2×2 and 3×3 buffers. However, the superior results of simulated annealing over RL (all masks) on the 15-color problems with 7×7 and 8×8 buffers are not significant. There are significant performance differences with $p < 0.01$ between RL (all masks) and RL (no masks) on problems with 4×4–8×8 buffers. The differences between RL (all masks) and RL (invalid mask) are also significant with $p < 0.01$ on problems with 3×3–8×8 buffers. We even find a significant difference on the 15-color problems with 8×8 buffers between RL (all masks) and RL (invalid + greedy retrieval). Again, the differences between RL (all masks) and RL (invalid + greedy retrieval + fast-track) are not significant.

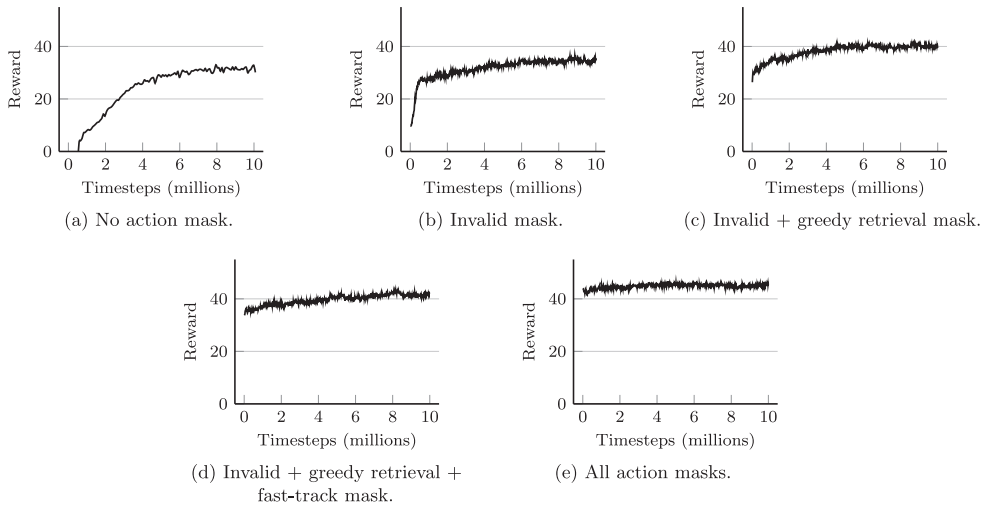


Fig. 4. Learning curves for RL models with 10 colors and 4 × 4 buffer and varying action masks.

In summary, action masking significantly increased the performance of RL in the paint shop problem depending on the problem size. We considered different action masks that enforce provably optimal actions (greedy retrieval + fast-track), as well as reasonable actions (greedy storage). The largest marginal performance benefits could be observed when enforcing greedy retrieval. Enforcing fast-track actions had a smaller influence, probably since fast-track actions are often not possible as they require empty buffer lanes.

We analyze the learning curves of all combinations of action masks for the setting with 10 colors and 4 × 4 buffer as shown in Fig. 4. The plots show that the RL policies trained with more action masks converge to a greater reward. Furthermore, the learning process is faster if more action masks are added to policy learning. Specifically, the model without action masking only reaches positive rewards after 500,000 time steps. Conversely, including the invalid mask ensures positive rewards from the first learning episode. Increasing the number of action masks leads to a higher initial reward over the first learning episodes. Therefore, action masking not only leads to higher overall rewards but also mitigates the "cold start" problem of an untrained RL policy.

4. Problem 2: Peak load management

A peak load management system (LMS) needs to decide when to turn off electric devices with high energy consumption to keep the peak load below a given threshold (Dong et al., 2017; Papier, 2016). Violations of this threshold must be avoided at any time point as they entail large payments to the energy supplier.

4.1. Environment

The company can turn off large energy consumers like air conditioning to reduce the peak load. Hence, the action space consists of only two actions: turning off air conditioning and leaving it on

$$A^{\text{LMS}} = \{\text{off}, \text{on}\}. \tag{20}$$

If the air conditioning is turned off, energy consumption is zero during the next time period.

We consider a time horizon of one day, while one period lasts 15 min. This yields a finite time horizon of $T = 24 \times 4 = 96$ time steps. However, due to legal requirements, executing action `off` to turn off air conditioning is constrained to a total of three executions per day. Therefore, action `off` should only be performed when a high peak load occurs.

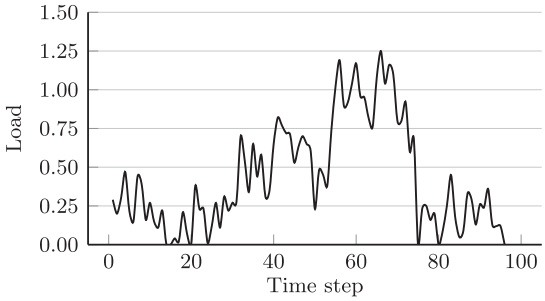


Fig. 5. Load curve for load management system over 96 timesteps.

The system's load curve is illustrated in Fig. 5. While this curve remains constant throughout the study, it is initially unknown to the reinforcement learning policy and must be discovered during training. We do not evaluate different or changing curves, as the setup with a fixed curve and noisy forecasts already creates sufficient complexity for analyzing the policy's learning capabilities.

The state of the LMS is given by the actual consumption c_{t-1} of the last time period, the estimated consumption $\widehat{c}_t$ of the next time period, and how often the action `off` can still be performed, denoted by n

$$s_t^{\text{LMS}} = (c_{t-1}, \widehat{c}_t, n). \tag{21}$$

The estimated consumption is given by a predictive model $\widehat{c}_t = c_t + \varepsilon$ with normally distributed error $\varepsilon \sim \mathcal{N}(0, \sigma)$ and standard deviation σ.

The goal of the LMS problem is to keep the maximum load over all time steps below a given threshold ζ. The RL policy's actions to reduce the peak load are successful if the maximum load $c^* = \max_{t=0}^{T-1}\{c_t \mid a_t = \text{on}\}$ over all $T = 96$ time steps is lower than the peak load threshold of $\zeta = 1.24$. Therefore, whether or not the RL policy was successful in managing the peak load can only be determined at the end of the time horizon. Accordingly, we define the following reward function

$$R^{\text{LMS}}(a, s_t) = \begin{cases} 1 & \text{if } t = T-1 \wedge c^* < \zeta \\ -1 & \text{if } t = T-1 \wedge c^* \geq \zeta \\ 0 & \text{if } t < T-1. \end{cases} \tag{22}$$

4.2. Action masking

We consider action masks that restrict the execution of action `off`. Given that `off` can only be performed a limited number of times due to legal constraints, it seems reasonable to perform it only, if the estimate

Table 1
Fraction of solved instances in percent from 1000 episodes after policy learning for one million time steps for different action mask thresholds and noise levels.

Noise level σ	Action mask threshold θ^{LMS}						
	none	0.20	0.40	0.60	0.80	1.00	1.20
0	60.0 ± 51.6	50.0 ± 52.7	80.0 ± 42.2	20.0 ± 42.2	40.0 ± 51.6	60.0 ± 51.6	100.0 ± 0.0
0.1	36.5 ± 17.7	50.6 ± 25.8	36.4 ± 2.2	52.3 ± 22.7	41.7 ± 16.2	58.1 ± 30.8	66.0 ± 0.0
0.2	29.9 ± 20.8	33.9 ± 16.3	37.5 ± 16.0	33.7 ± 14.7	40.1 ± 21.2	60.1 ± 23.1	49.1 ± 4.3
0.3	26.1 ± 30.6	16.3 ± 7.4	23.8 ± 12.0	19.1 ± 8.0	40.2 ± 28.3	53.6 ± 20.4	43.6 ± 7.4
0.4	11.1 ± 12.6	12.0 ± 5.4	10.3 ± 5.2	13.2 ± 6.6	13.0 ± 3.3	28.8 ± 20.3	43.9 ± 4.2

Table 2
Number of seeds (out of 10) that resulted in a stable policy (success rate ≥ 25%).

Noise level σ	Action mask threshold θ^{LMS}						
	None	0.20	0.40	0.60	0.80	1.00	1.20
none	6/10	5/10	8/10	2/10	4/10	6/10	10/10
0.10	6/10	9/10	10/10	9/10	9/10	8/10	10/10
0.20	6/10	5/10	6/10	6/10	6/10	10/10	10/10
0.30	4/10	0/10	3/10	1/10	6/10	8/10	10/10
0.40	1/10	0/10	0/10	0/10	0/10	4/10	10/10

of the next period $\hat{c}_t$ is sufficiently high. Therefore, we define the mask with respect to $\hat{c}_t$, so that `off` can only be performed if the predicted consumption is above a given threshold

$$m(a, s_t) = \begin{cases} 1 & \text{if } \hat{c}_t \geq \theta^{LMS} \text{ and } a = \text{off} \\ 1 & \text{if } a = \text{on} \\ 0 & \text{else.} \end{cases} \tag{23}$$

We consider multiple masks with varying θ^{LMS} between 0.20 and 1.20. The action masks become more restrictive for higher values of θ^{LMS}.

4.3. Results

In our evaluation, we vary the noise level σ and the action mask threshold θ^{LMS}. For each configuration, we train ten RL policies with ten different model seeds for one million time steps and report the success rates over 1000 fixed instances to account for randomness in $\hat{c}_t$.

Table 1 reports the mean success rates over 10 independently trained seeds per configuration. Success rates broadly increase with the mask threshold θ^{LMS}. At σ = none, $\theta^{LMS} = 1.20$ reaches 100%. At $\sigma = 0.10$, it reaches 66%, compared to 33% for unmasked RL. But the standard deviation across seeds is high: up to 52.7 percentage points at σ = none, $\theta^{LMS} = 0.20$, and still approximately 30 points for several thresholds at $\sigma = 0.10$–0.30. Individual mean values should hence be read with caution. The relationship between threshold and success rate is also not strictly monotonic. At $\sigma = 0.20$ and $\sigma = 0.30$, $\theta^{LMS} = 1.00$ slightly outperforms the stricter $\theta^{LMS} = 1.20$ (55% vs. 45% at $\sigma = 0.20$; 50% vs. 39% at $\sigma = 0.30$). An overly restrictive mask can thus even become counter-productive once the load estimate is noisy enough. Across all thresholds, success rates decline with noise, from 100% at σ = none to at most 36% at $\sigma = 0.40$. This reflects the growing difficulty of the prediction task. Overall, action masking remains necessary to learn an effective policy at all: across every noise level $\sigma \geq 0.10$, unmasked RL and weakly masked RL ($\theta^{LMS} \leq 0.80$) never reach the success rates achieved by $\theta^{LMS} \geq 1.00$.

We perform pairwise two-proportion z-tests with Holm correction between all action mask thresholds based on 1000 evaluation episodes per model. Of the 105 pairwise comparisons across all noise levels, 81 are significant at $p_{\text{Holm}} < 0.05$. $\theta^{LMS} = 1.20$ differs significantly from every other threshold in 29 of its 30 pairwise comparisons; the sole exception is $\theta^{LMS} = 0.80$ at $\sigma = 0.30$ ($p_{\text{Holm}} = 0.303$), where the two thresholds achieve similar success rates (40% vs. 44%). This confirms that the advantage of stricter action masking is statistically robust. The complete test results are provided in Appendix B.

Due to the high variance across seeds, we also consider the number of seeds (out of 10) that learned a stable policy, defined as a success rate of at least 25%, which is well above chance. The results are shown in Table 2. We observe a clear pattern: only $\theta^{LMS} = 1.20$ reaches full reliability, 10 of 10 seeds, at every noise level from σ = none to $\sigma = 0.40$. No other threshold does. $\theta^{LMS} = 1.00$ drops to 4 of 10 stable seeds at the highest noise level. Every threshold $\theta^{LMS} \leq 0.80$ falls to 0 or 1 of 10 stable seeds for at least one noise level $\sigma \geq 0.30$. Action masking is therefore close to a necessary condition for reliable policy learning in this problem with constrained actions.

We also consider the learning curves for the noise levels $\sigma = 0.20, 0.40$ and θ^{LMS} = none, 0.40, 0.80, 1.20 as shown in Fig. 6. We always plot the learning curve of the model with the first seed. Higher thresholds learn faster and show a more efficient learning process. $\theta^{LMS} = 1.20$ rises from −1.0 to near zero within 200,000 steps, while the weaker thresholds ($\theta^{LMS} \leq 0.80$) stay near −1.0 for several hundred thousand steps and then plateau lower and noisier, often dipping back towards −1.0. This holds at both noise levels, but is more pronounced at $\sigma = 0.40$. Finally, we note that no configuration reaches a positive reward on average.

5. Problem 3: Inventory management

The objective of the inventory management problem is to minimize the cost of operating an inventory (e.g., Boute et al., 2022; Heger & Klein, 2024). At each time step, a stochastic demand is realized and the operating agent places an order to replenish the inventory. However, orders do not arrive instantly as they are delayed by the lead time, which can be either stochastic or deterministic. The total costs of operating the inventory are the sum of inventory and lost sales costs.[2] There is a trade-off between minimizing losing sales and minimizing inventory holding costs. One major challenge of the inventory problem is to learn the distribution of demand and lead times. However, due to its intensive study, several useful heuristics were developed in prior works (Perera & Sethi, 2023; Zipkin, 2008), which we consider for action masking.

5.1. Environment

We adopt the RL formalization of the inventory management problem by Gijsbrechts et al. (2022) and adjust it to include stochastic lead times. An overview of all parameters and variables is provided in Table 3. The state of the system is given by the current inventory and the pipeline vector of incoming orders

$$s_t^{\text{IM}} = (\underbrace{I_t}_{\substack{\text{Current} \\ \text{inventory}}}, \underbrace{O_{t,1}, \dots O_{t,N}}_{\substack{\text{Pipeline vector of} \\ \text{incoming orders}}}). \tag{24}$$

The pipeline vector has length N, denoting the maximum lead time.

[2] We consider the lost sales setting instead of backlogged systems since it is more challenging and interesting. In fact, the so called base-stock policy is proven to be optimal for backlogged systems (Zipkin, 2008).

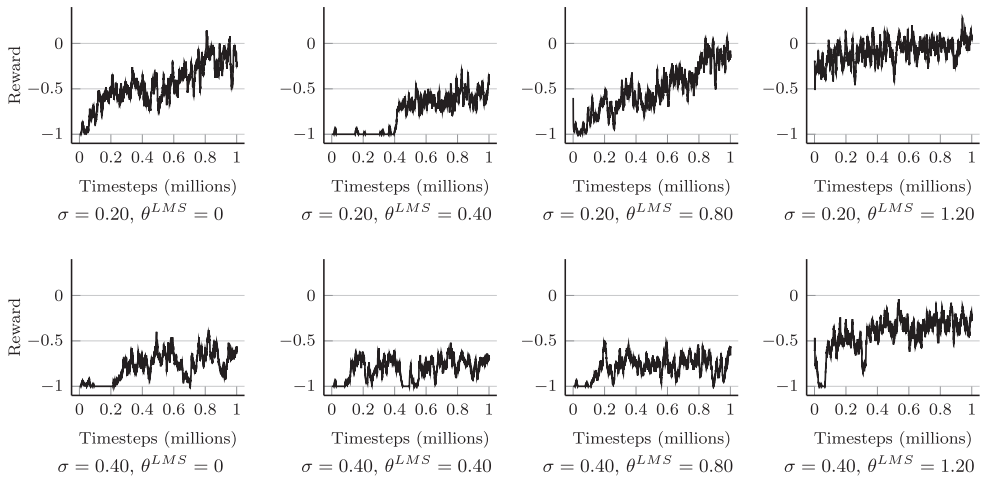


Fig. 6. Learning curves for RL models with $\sigma = 0.20, 0.40$ and thresholds $\theta^{LMS} = 0$, 0.40, 0.80, 1.20.

Table 3
Overview of parameters and variables.

Parameters	
$c = 1$	Holding cost
$p \in \{1, 4, 9\}$	Lost sales cost
$N \in \{4, 8\}$	Maximum lead time: $N = 4$ (deterministic) or $N = 8$ (stochastic)
$\lambda = 5$	Mean demand of Poisson distribution
$H = 5000$	Time horizon
$\Delta = 1$	Discretization value of action space
Variables	
I_t	Inventory at time t
$Q_{t,i}$	Pipeline vector at time t and position $i \in \{1, \dots N\}$
d_t	Demand at time t
L_t	Lead time at time t: deterministic ($L_t = N$) or stochastic $L_t \sim \text{Unif}(1, N)$

The actions specify the amount of items to be reordered. We discretize the action space by only allowing quantities given as multiples of $\Delta = 1$

$$A^{\text{IM}} = \{0, \Delta, 2\Delta, \dots, 10\Delta\}. \tag{25}$$

After a certain quantity is ordered, the next incoming order is added to the inventory level and the orders of the pipeline vector are shifted one step forward

$$I'_t = I_t + Q_{t,1}, \quad Q'_{t,i} = Q_{t,i+1}, \quad Q_{t,N} = 0. \tag{26}$$

Next, the demand d_t is drawn from a Poisson distribution

$$d_t \sim \text{Pois}(\lambda) \tag{27}$$

with mean λ and subtracted from the inventory level. If the demand is greater than the inventory level, the inventory is set to zero and the remaining demand causes lost sales costs

$$I_{t+1} = (I_t + Q_{t,1} - d_t)^+. \tag{28}$$

Lastly, the selected action a is executed by sampling a lead time L_t from the discrete uniform distribution and adding the ordered quantity a to the pipeline vector at the respective lead time

$$L_t \sim \text{Unif}(1, N) \tag{29}$$

$$Q_{t+1,i} = \begin{cases} Q'_{t,i} + a & \text{if } i = L_t \\ Q'_{t,i} & \text{else.} \end{cases} \tag{30}$$

We also evaluate deterministic lead times with $L_t = 4$.

The reward function depends on whether or not sales are lost. If orders cannot be met, the agent receives a penalty proportionate to the number of lost sales. Otherwise, holding costs are charged proportionate to the inventory

$$R^{\text{IM}}(a, s_t) = \begin{cases} -c(I_t + Q_{t,1} - d_t) & \text{if } I_t + Q_{t,1} - d_t \geq 0 \\ p(I_t + Q_{t,1} - d_t) & \text{else.} \end{cases} \tag{31}$$

Therefore, maximizing the cumulative reward over the given time horizon H is equivalent to minimizing the total cost.

5.2. Action masking

Our action masks are informed by the *base stock heuristic* (e.g., Bijvank & Vis, 2011; Huh et al., 2009; Zipkin, 2008). This heuristic depends on the parameter S, the *base-stock level* and it orders $h(s_t) = S - I_t - \sum_i Q_{t,i}$, which sets the sum of inventory and pipeline to the base-stock level. The heuristic is employed independently of the lost sales costs p. However, the heuristic converges to the optimal policy for $p \to \infty$ (Huh et al., 2009).

We derive three action masks based on this heuristic, $m^{\text{INT},1}$, $m^{\text{INT},2}$, and m^{THR}. The first two masks $m^{\text{INT},1}$ and $m^{\text{INT},2}$ allow actions that order a similar quantity as the prescribed action that deviates at most one or two Δ from the prescribed action.

$$m^{\text{INT},1}(a, s_t) = \begin{cases} 1 & \text{if } |a - (S - I_t - \sum_i Q_{t,i})| \leq \Delta \\ 0 & \text{else.} \end{cases} \tag{32}$$

$$m^{\text{INT},2}(a, s_t) = \begin{cases} 1 & \text{if } |a - (S - I_t - \sum_i Q_{t,i})| \leq 2\Delta \\ 0 & \text{else.} \end{cases} \tag{33}$$

The third action mask m^{THR} uses the prescribed action $h(s_t)$ as a threshold and allows all actions that order at least as much

$$m^{\text{THR}}(a, s_t) = \begin{cases} 1 & \text{if } a + I_t + \sum_i Q_{t,i} \geq S \\ 0 & \text{else.} \end{cases} \tag{34}$$

For a given instance of the inventory problem, the optimal base-stock level S can be determined by a search algorithm. The optimal values have already been computed for many settings and can be found in several sources (e.g., Huh et al., 2009). In our setting, the optimal values are given as $S = 18$ for $p = 1$, $S = 25$ for $p = 4$, and $S = 28$ for $p = 9$.

5.3. Results

We compare the average inventory cost for $m^{\text{INT},1}$, $m^{\text{INT},2}$, m^{THR}, no action mask, and the pure *base stock* heuristic. We evaluate deterministic and stochastic lead times and lost sales costs of $p \in \{1, 4, 9\}$ with fixed holding costs $c = 1$. We only consider small values of p as the base stock heuristic becomes optimal for larger p. For each configuration, we

Table 4
Results of inventory problem.

Lost sales cost p	Lead time	Action mask	Average costs	Cost difference
1	Deterministic	None	$\mathbf{2.102 \pm 0.007}$	–
1	Deterministic	$m^{\mathrm{INT},1}$	2.144 ± 0.006	2.0%
1	Deterministic	$m^{\mathrm{INT},2}$	2.128 ± 0.013	1.2%
1	Deterministic	m^{THR}	2.113 ± 0.035	0.5%
1	Deterministic	Base-stock heuristic	2.378 ± 0.000	13.1%
1	Stochastic	None	$\mathbf{3.186 \pm 0.008}$	–
1	Stochastic	$m^{\mathrm{INT},1}$	3.430 ± 0.014	7.7%
1	Stochastic	$m^{\mathrm{INT},2}$	3.324 ± 0.005	4.3%
1	Stochastic	m^{THR}	3.516 ± 0.010	10.4%
1	Stochastic	Base-stock heuristic	3.627 ± 0.000	13.8%
4	Deterministic	None	$\mathbf{4.858 \pm 0.042}$	–
4	Deterministic	$m^{\mathrm{INT},1}$	4.950 ± 0.027	1.9%
4	Deterministic	$m^{\mathrm{INT},2}$	4.867 ± 0.017	0.2%
4	Deterministic	m^{THR}	5.122 ± 0.025	5.4%
4	Deterministic	Base-stock heuristic	5.212 ± 0.000	7.3%
4	Stochastic	None	7.946 ± 0.022	0.2%
4	Stochastic	$m^{\mathrm{INT},1}$	8.094 ± 0.014	2.0%
4	Stochastic	$m^{\mathrm{INT},2}$	$\mathbf{7.934 \pm 0.028}$	–
4	Stochastic	m^{THR}	8.051 ± 0.008	1.5%
4	Stochastic	Base-stock heuristic	8.213 ± 0.000	3.5%
9	Deterministic	None	7.170 ± 0.210	1.7%
9	Deterministic	$m^{\mathrm{INT},1}$	7.167 ± 0.033	1.7%
9	Deterministic	$m^{\mathrm{INT},2}$	$\mathbf{7.047 \pm 0.033}$	–
9	Deterministic	m^{THR}	7.325 ± 0.020	3.9%
9	Deterministic	Base-stock heuristic	7.304 ± 0.000	3.6%
9	Stochastic	None	$\mathbf{11.682 \pm 0.079}$	–
9	Stochastic	$m^{\mathrm{INT},1}$	12.222 ± 0.038	4.6%
9	Stochastic	$m^{\mathrm{INT},2}$	11.859 ± 0.037	1.5%
9	Stochastic	m^{THR}	11.765 ± 0.034	0.7%
9	Stochastic	Base-stock heuristic	12.387 ± 0.000	6.0%

Note: Average costs are provided as the mean ± standard deviation over five models with different training seeds. Each model is evaluated on 100 fixed instances, each with time horizon 5000 steps.

train an RL policy for 1 million time steps and average the inventory costs over 100 evaluations.

Table 4 provides the results. We observe a cost-dependent pattern. For low lost-sales costs ($p = 1$), unmasked RL achieves the lowest average cost in both the deterministic and stochastic setting, outperforming all masked variants and the base-stock heuristic by a substantial margin (up to 13.8%). Here, action masking imposes a bias towards the base-stock policy that unconstrained RL does not need, since the optimal ordering behavior at low p deviates from what the base-stock heuristic prescribes. As p increases, this picture reverses: the cost gap of the base-stock heuristic to the best method shrinks from over 13% at $p = 1$ to 3.5%–7.3% at $p = 4$ and $p = 9$, consistent with its known asymptotic optimality as $p \to \infty$. Correspondingly, the masked variants become competitive and, in several configurations, outperform unmasked RL outright: $m^{\mathrm{INT},2}$ achieves the lowest cost among all methods at $p = 4$ (stochastic) and $p = 9$ (deterministic), and comes within 1.5% of unmasked RL at $p = 9$ (stochastic). Between the two masking strategies, m^{INT} consistently outperforms m^{THR}, and the wider band $m^{\mathrm{INT},2}$ tends to outperform the narrower $m^{\mathrm{INT},1}$ as p grows, suggesting that some flexibility around the heuristic's suggested order quantity remains beneficial even as the heuristic itself becomes a stronger anchor. Overall, these results illustrate the bias–variance trade-off central to this study: action masking helps precisely when the underlying heuristic is close to optimal, and hurts when it is not.

To assess whether the differences in inventory costs are statistically significant, we perform one-way ANOVAs followed by pairwise Welch's t-tests (Welch, 1947) with Holm correction (Holm, 1979) for each configuration. The one-way ANOVA based on aggregated summary statistics suggest $p < 0.001$ for each configuration. In addition, the vast majority of pairwise comparisons are significant with $p < 0.05$ or even $p < 0.001$. Non-significant differences in the deterministic setting are m^{THR} vs no masking for $p = 1$, $m^{\mathrm{INT},2}$ vs no masking for $p = 4$, $m^{\mathrm{INT},1}$ vs no masking and m^{THR} vs base stock heuristic for $p = 9$. In the stochastic setting, the only non-significant difference is $m^{\mathrm{INT},2}$ vs no masking for $p = 4$. This observation is explained by the small standard deviations of cumulative inventory costs. The standard deviations over the 100 evaluation runs across all considered approaches range from 137.8 to 1393.3 (mean = 485.4). Given the long time horizon ($H = 5000$), even moderate per-period differences accumulate to substantial differences in total costs, resulting in large test statistics and correspondingly small p-values. The complete test results are provided in Appendix B of the supplementary material.

We also consider the learning curves for stochastic lead times and different lost sales costs in Fig. 7. We always plot the learning curve of the model with the first seed. We observe that the RL models without action masks and with m^{THR} require more timesteps to reach a competitive performance, which is expected by design. Conversely, the RL models with $m^{\mathrm{INT},1}$ and $m^{\mathrm{INT},2}$ start from a higher initial reward as they force the policy to perform actions similar to the base stock heuristic. Consistent with our prior analysis, the models without action mask and $m^{\mathrm{INT},2}$ achieve the highest rewards.

Note that the learning curves for the inventory optimization problem tend to be much smoother compared to those for the paint shop and load management problems. This can be attributed to the more predictable dynamics of the inventory optimization problem. In the load management problem, only a single non-zero reward is given at the end of each episode, and its value reflects the cumulative effect of all previous actions. A small change in any action can have a significant impact on the final reward, leading to more fluctuations in the reward. Similarly, in the paint shop problem, individual actions can have persistent long-term effects; for example, a poor storage operation can block a buffer lane for many timesteps with detrimental effects on subsequent actions. By contrast, in the inventory management problem, the total reward is more gradually affected by individual actions. For

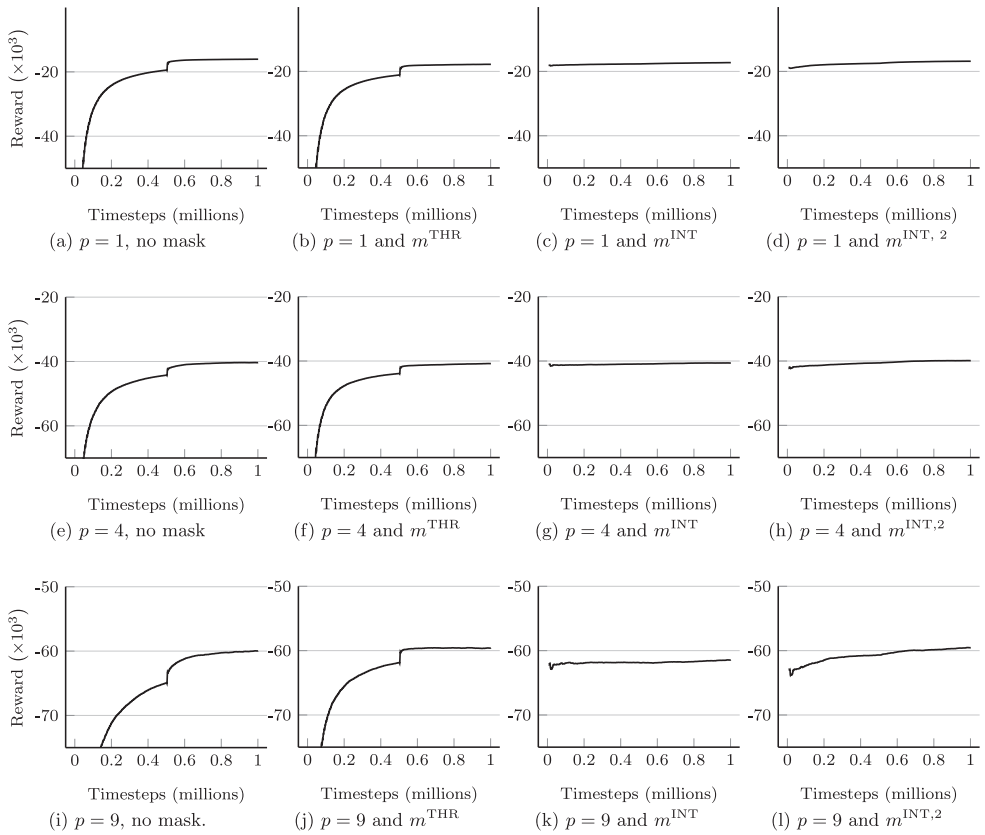


Fig. 7. Learning curves for RL models (first seed) with stochastic lead times.

instance, ordering 7 units instead of 6 causes only a small change to the total reward over the entire time horizon, leading to a smoother reward function and, consequently, smoother learning curves.

6. Conclusion

We proposed the integration of human domain knowledge into reinforcement learning via action masking. Based on three OR problems, we showed that in addition to excluding invalid actions, action masking can also be employed to (i) prescribe heuristics and (ii) enforce provably optimal actions. In particular, our results suggest that the performance of the trained policy can be significantly improved. For problems with constrained action spaces, action masking can help to prevent unreasonable executions of the constrained action. As such, action masking may not only improve sample efficiency and solution quality. Instead, it might also be a necessary requirement for learning stable and effective policies. However, at the same time, we also found that action masking can decrease the performance if prescribing overly strict non-optimal heuristics.

Our study adds to the literature that focuses on solving OR problems with RL. Specifically, we propose action masking as a method to integrate human domain knowledge during policy training and application. So far, related works employed action masking to disallow invalid actions (e.g., Huang & Ontañón, 2022; Luo et al., 2022; Wang et al., 2024). We go beyond excluding invalid actions by guiding the policy towards (i) optimal actions and (ii) other reasonable actions suggested by near-optimal heuristics. Thereby, we aim to improve sample efficiency and the resulting policy performance, as well as adoption of the model in practice (Notz & Pibernik, 2024).

From a theoretical view, enforcing optimal actions should not necessarily change the resulting policy given a sufficient number of training timesteps. Yet, we found significant differences in the solution quality between RL policies with and without enforcement of optimal actions, even after millions of timesteps. This empirical observation is explained by the fact that DRL does not provide any guarantees regarding convergence to a global optimum (Sutton & Barto, 2018). Employing action masking to enforce optimal actions, if applicable, might hence not only increase sample efficiency during training but also the final policy quality.

However, incorporating human domain knowledge in the form of (non-optimal) heuristics does not come without limitations. The key challenge is to choose the appropriate restrictiveness of the action mask in light of the implied bias–variance trade-off: if the heuristics themselves are imperfect, restrictive action masks lower variance while increasing bias in the solution quality. Conversely, a less restrictive mask increases variance but allows more flexibility for learning superior or even optimal policies. Future research should hence explore methods to dynamically adjust or refine action masks as the agent learns, ensuring reasonable trade-offs between following heuristic human domain knowledge and allowing flexibility to explore alternative actions.

A major challenge for future research is to identify the categories of OR problems that are most suitable for heuristic-based action masking. We argue that two characteristics are particularly important. First, the problem should allow for defining action masks while still preserving sufficient flexibility for exploration. This typically requires the action space of the MDP formulation to exhibit a meaningful structure, such that actions can be ranked according to relevant problem characteristics. For example, actions may naturally correspond to inventory order quantities, machine assignments, or scheduling decisions. Even if such a structure is not explicitly given, it may sometimes be constructed. In scheduling problems, for instance, jobs could be ranked according to their processing times, due dates, or frequency, thereby enabling structured action masks.

Second, useful human domain knowledge must be available. Action masking relies on heuristics that identify actions which are unlikely to

contribute to high-quality solutions while still leaving sufficient alternative actions for the RL agent. Such heuristics are particularly common for well-established combinatorial optimization problems that have been studied extensively over decades. Examples include dispatching rules for the Job Shop Scheduling Problem, priority rules for production scheduling, inventory replenishment policies, and constructive heuristics for bin packing. These heuristics provide valuable prior knowledge that can be translated into action masks while still allowing the RL agent sufficient flexibility to improve upon them.

Taken together, heuristic-based action masking is expected to be most effective for OR problems that combine (i) discrete and structured action spaces with (ii) reliable human domain knowledge that can be translated into flexible action masks. In contrast, its advantages are likely to diminish for problems with continuous or weakly structured action spaces and limited expert knowledge. In such settings, action masks may either become overly restrictive, excluding actions that are ultimately required for high-quality solutions, or overly conservative, providing little reduction of the search space.

From a practical view, action masking can also enhance the safety of RL systems in real-world applications. By constraining the agent's actions to safe and permissible options, organizations can ensure that the system operates within predefined boundaries, reducing the risk of unintended or harmful behaviors. While a randomly initialized policy first needs to learn the basic rules of the environment, masked RL performs reasonable actions already in early stages of learning. This is particularly important in safety-critical domains, such as autonomous driving, banking, and healthcare. Businesses can also rely on action masking for post hoc policy adjustments. If human experts identify non-optimal actions that should be included or excluded, they simply need to define a novel action mask and apply it to a trained policy instead of retraining the policy. Businesses could also define separate action masks for different departments, products, and factories, further increasing the utility of a trained RL policy.

An alternative to action masking is given by reward shaping (Cheng et al., 2021). Here, the reward function can be defined in a way that guides the RL policy towards heuristic actions. Reward shaping is less restrictive regarding how the policy evolves. An action receiving a large positive reward might not even be chosen if the policy never explores it. In addition, the higher reward for an action might not reach an overall higher cumulative reward over the entire learning episode. Conversely, action masking directly enforces the suggested action(s) in specific states, which restricts the policy from exploring potential alternatives. We therefore encourage future studies to look deeper into the pros and cons of action masking vs. reward shaping.

Finally, future research should empirically examine whether incorporating human knowledge via action masking increases the acceptance and actual usage of RL-based decision support systems. Controlled experiments or field studies could compare systems with and without masking to assess differences in adoption, trust, and reliance. This would help validate whether the proposed benefits extend beyond technical performance to practical deployment, ultimately paving the way for RL to become a widely adopted and integral component of decision-making in real-world operations.

CRediT authorship contribution statement

Mirko Stappert: Writing – original draft, Software, Methodology, Formal analysis, Conceptualization. **Bernhard Lutz:** Writing – review & editing, Writing – original draft, Conceptualization. **Niklas Goby:** Software, Conceptualization. **Dirk Neumann:** Supervision.

Declaration of generative AI in scientific writing:

During the preparation of this work the authors used ChatGPT-5.5 in order to improve grammar and clarity. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding declaration:

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cie.2026.112340.

Data availability

A replication package including Python code and problem instances is provided at https://doi.org/10.6084/m9.figshare.33098051.

References

Arora, S., & Doshi, P. (2021). A survey of inverse reinforcement learning: Challenges, methods and progress. *Artificial Intelligence*, *297*, Article 103500.

Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, *290*(2), 405–421.

Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., et al. (2019). Dota 2 with large scale deep reinforcement learning. ArXiv preprint https://arxiv.org/abs/1912.06680.

Bianchi, R. A., Celiberto Jr, L. A., Santos, P. E., Matsuura, J. P., & De Mantaras, R. L. (2015). Transferring knowledge as heuristics in reinforcement learning: A case-based approach. *Artificial Intelligence*, *226*, 102–121.

Bijvank, M., & Vis, I. F. (2011). Lost-sales inventory theory: A review. *European Journal of Operational Research*, *215*(1), 1–13.

Boute, R. N., Gijsbrechts, J., Van Jaarsveld, W., & Vanvuchelen, N. (2022). Deep reinforcement learning for inventory control: A roadmap. *European Journal of Operational Research*, *298*(2), 401–412.

Brammer, J., Lutz, B., & Neumann, D. (2022a). Permutation flow shop scheduling with multiple lines and demand plans using reinforcement learning. *European Journal of Operational Research*, *299*(1), 75–86.

Brammer, J., Lutz, B., & Neumann, D. (2022b). Stochastic mixed model sequencing with multiple stations using reinforcement learning and probability quantiles. *OR Spectrum*, *44*(1), 29–56.

Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., et al. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. *Transactions on Machine Learning Research*.

Chen, W., Zhang, Z., Tang, D., Liu, C., Gui, Y., Nie, Q., & Zhao, Z. (2024). Probing an lstm-ppo-based reinforcement learning algorithm to solve dynamic job shop scheduling problem. *Computers & Industrial Engineering*, *197*, Article 110633.

Cheng, C.-A., Kolobov, A., & Swaminathan, A. (2021). Heuristic-guided reinforcement learning. *Advances in Neural Information Processing Systems*, *34*, 13550–13563.

Choi, T.-M., Kumar, S., Yue, X., & Chan, H.-L. (2022). Disruptive technologies and operations management in the industry 4.0 era and beyond. *Production and Operations Management*, *31*(1), 9–31.

Dietvorst, B. J., Simmons, J. P., & Massey, C. (2015). Algorithm aversion: people erroneously avoid algorithms after seeing them err.. *Journal of Experimental Psychology: General*, *144*(1), 114–126.

Dietvorst, B. J., Simmons, J. P., & Massey, C. (2018). Overcoming algorithm aversion: People will use imperfect algorithms if they can (even slightly) modify them. *Management Science*, *64*(3), 1155–1170.

Dong, C., Ng, C. T., & Cheng, T. (2017). Electricity time-of-use tariff with stochastic demand. *Production and Operations Management*, *26*(1), 64–79.

Esteso, A., Peidro, D., Mula, J., & Díaz-Madroñero, M. (2023). Reinforcement learning applied to production planning and control. *International Journal of Production Research*, *61*(16), 5772–5789.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, *32*(200), 675–701.

Gijsbrechts, J., Boute, R. N., Van Mieghem, J. A., & Zhang, D. J. (2022). Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, *24*(3), 1349–1368.

Goby, N., Brandt, T., & Neumann, D. (2023). Deep reinforcement learning with combinatorial actions spaces: An application to prescriptive maintenance. *Computers & Industrial Engineering*, *179*, Article 109165.

Griffith, S., Subramanian, K., Scholz, J., Isbell, C. L., & Thomaz, A. L. (2013). Policy shaping: Integrating human feedback with reinforcement learning. In C. Burges, L. Bottou, M. Welling, Z. Ghahramani, & K. Weinberger (Eds.), *volume 26*, *Advances in neural information processing systems*. Curran Associates, Inc..

Heger, J., & Klein, R. (2024). Assortment optimization: A systematic literature review. *OR Spectrum*, *46*(4), 1099–1161.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2), 65–70.

Hou, Y., Liao, X., Chen, G., & Chen, Y. (2025). Co-evolutionary nsga-iii with deep reinforcement learning for multi-objective distributed flexible job shop scheduling. *Computers & Industrial Engineering*, *203*, Article 110990.

Huang, S., & Ontañón, S. (2022). A closer look at invalid action masking in policy gradient algorithms. *vol. 35*, In *The international FLAIRS conference proceedings*.

Huh, W. T., Janakiraman, G., Muckstadt, J. A., & Rusmevichientong, P. (2009). Asymptotic optimality of order-up-to policies in lost sales inventory systems. *Management Science*, *55*(3), 404–420.

Hussein, A., Gaber, M. M., Elyan, E., & Jayne, C. (2017). Imitation learning: A survey of learning methods. *ACM Computing Surveys*, *50*(2), 1–35.

Kaufmann, T., Weng, P., Bengs, V., & Hüllermeier, E. (2023). A survey of reinforcement learning from human feedback. ArXiv preprint https://https://arxiv.org/abs/2312.14925.

Kayhan, B. M., & Yildiz, G. (2023). Reinforcement learning applications to machine scheduling problems: A comprehensive literature review. *Journal of Intelligent Manufacturing*, *34*(3), 905–929.

Lehmann, C. A., Haubitz, C. B., Fügener, A., & Thonemann, U. W. (2022). The risk of algorithm transparency: How algorithm complexity drives the effects on the use of advice. *Production and Operations Management*, *31*(9), 3419–3434.

Luo, P. C., Xiong, H. Q., Zhang, B. W., Peng, J. Y., & Xiong, Z. F. (2022). Multi-resource constrained dynamic workshop scheduling based on proximal policy optimisation. *International Journal of Production Research*, *60*(19), 5937–5955.

Nemenyi, P. B. (1963). Distribution-free multiple comparisons. (Ph.D. thesis), Princeton University.

Notz, P. M., & Pibernik, R. (2024). Explainable subgradient tree boosting for prescriptive analytics in operations management. *European Journal of Operational Research*, *312*(3), 1119–1133.

Ogunfowora, O., & Najjaran, H. (2023). Reinforcement and deep reinforcement learning-based solutions for machine maintenance planning, scheduling policies, and optimization. *Journal of Manufacturing Systems*, *70*, 244–263.

Panzer, M., & Bender, B. (2022). Deep reinforcement learning in production systems: A systematic literature review. *International Journal of Production Research*, *60*(13), 4316–4341.

Papier, F. (2016). Managing electricity peak loads in make-to-stock manufacturing lines. *Production and Operations Management*, *25*(10), 1709–1726.

Paraschos, P. D., & Koulouriotis, D. E. (2024). Learning-based production, maintenance, and quality optimization in smart manufacturing systems: A literature review and trends. *Computers & Industrial Engineering*, *198*, Article 110656.

Perera, S. C., & Sethi, S. P. (2023). A survey of stochastic inventory models with fixed costs: Optimality of (s, s) and (s, s)-type policies—discrete-time case. *Production and Operations Management*, *32*(1), 131–153.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, *22*, 1–8.

Rolf, B., Jackson, I., Müller, M., Lang, S., Reggelin, T., & Ivanov, D. (2023). A review on reinforcement learning algorithms and applications in supply chain management. *International Journal of Production Research*, *61*(20), 7151–7179.

Schiewe, P., & Stinzendörfer, M. (2024). Optimizing combined tours: The truck-and-cargo-bike case. *OR Spectrum*, *46*(2), 545–587.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, *362*, 1140–1144.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. (2017). Mastering the game of go without human knowledge. *Nature*, *550*, 354–359.

Stappert, Mirko, Lutz, Bernhard, Brammer, Janis, & Neumann, Dirk (2026). Solving the paint shop problem with flexible management of multi-lane buffers using reinforcement learning and action masking. *European Journal of Operational Research*, *332*(1), 52–65.

Sutton, R. S., & Barto, A. G. (2018). Introduction to reinforcement learning (2nd edition). Cambridge, MA: MIT Press.

Taylor, M. E., & Stone, P. (2009). Transfer learning for reinforcement learning domains: A survey.. *Journal of Machine Learning Research*, *10*, 1633–1685.

Valet, A., Altenmüller, T., Waschneck, B., May, M. C., Kuhnle, A., & Lanza, G. (2022). Opportunistic maintenance scheduling with deep reinforcement learning. *Journal of Manufacturing Systems*, *64*, 518–534.

Van Hasselt, H., Doron, Y., Strub, F., Hessel, M., Sonnerat, N., & Modayil, J. (2018). Deep reinforcement learning and the deadly triad. ArXiv preprint available at https://arxiv.org/abs/1812.02648.

Vinyals, O., Ewalds, T., Bartunov, S., Georgiev, P., Vezhnevets, A. S., Yeo, M., Makhzani, A., Küttler, H., Agapiou, J., Schrittwieser, J., et al. (2017). Starcraft II: A new challenge for reinforcement learning. ArXiv preprint https://arxiv.org/abs/1708.04782.

Wang, X., Zhang, L., Liu, Y., & Laili, Y. (2024). An improved deep reinforcement learning-based scheduling approach for dynamic task scheduling in cloud manufacturing. *International Journal of Production Research*, *62*(11), 4014–4030.

Weckenborg, C., Schumacher, P., Thies, C., & Spengler, T. S. (2024). Flexibility in manufacturing system design: A review of recent approaches from operations research. *European Journal of Operational Research*, *315*(2), 413–441.

Welch, B. L. (1947). The generalization of 'student's' problem when several different population variances are involved. *Biometrika*, *34*(1-2), 28–35.

Zhang, X., & Zhu, G.-Y. (2024). A literature review of reinforcement learning methods applied to job-shop scheduling problems. *Computers & Operations Research*, Article 106929.

Zipkin, P. (2008). Old and new methods for lost-sales inventory systems. *Operations Research*, *56*(5), 1256–1263.